EGSR: Complete Research Summary

Self-Distilling Reward Models from Asymmetric Information

Rohan Sikand · Stanford University

March 2026

Part I: The Idea

The Problem

RLVR trains LLMs on binary pass/fail. At every step, the environment produces rich execution feedback — stack traces, compiler errors, partial outputs — **all discarded** after computing the binary verdict.

Two failure modes of binary GRPO advantages $\hat{A}_i = (r_i - \mu)/\sigma$:

1. **All pass or all fail**: $\sigma = 0$, zero gradient signal
2. **No credit assignment**: $9/10$ correct $=$ same reward as random

Prior dense reward approaches: PRMs need expensive labels, external judges create distributional gap, self-rewarding LMs collapse without grounding.

The ingredients for dense reward already exist inside the training loop.

The Solution: EGSR

Same model π_θ, two roles, differing only in input:

$$\underbrace{\pi_\theta(\cdot \mid s)}_{\text{Student: generates}} \qquad \underbrace{\pi_\theta(\cdot \mid s, c_i, p_{\text{judge}})}_{\text{Teacher: evaluates given feedback}}$$

Dense reward via structured judgment classes:

$$R_{\text{self}}(s, a_i) = \sum_{k \in \mathcal{K}} w_k \cdot P_\theta(q = k \mid s, c_i, p_{\text{judge}})$$

Label-free maintenance:

$$\theta \leftarrow \theta - \eta \, \nabla_\theta [\mathcal{L}_{\text{GRPO}} + \lambda_1 \, \mathcal{L}_{\text{pred}} + \lambda_2 \, \mathcal{L}_{\text{cal}}]$$

The General Framework: Asymmetric Information

RLVR with execution feedback is one instantiation of a general principle:

Any time asymmetric post-hoc information exists, you can distill a reward model from the policy itself.

Setting	**Asymmetric info** I	$\mathcal{L}_{\text{pred}}$	**Status**
Code RLVR	Execution feedback	Predict stack traces	*This paper*
Math RLVR	Interpreter output	Predict errors	*This paper*
RLHF	Preference rationale	Predict human reaction	Future
Agents	Tool responses	Predict API output	Future
Formal verif.	Checker messages	Predict proof status	Future

The mechanism is invariant. The practitioner's question: **what post-hoc information do you have?**

Part II: Theoretical Contributions

Three Propositions (General, Not RLVR-Specific)

Proposition 1: Gradient variance reduction

Binary: $\mathrm{Var}[\hat{A}] \propto p(1-p)/G$ — vanishes at extreme p

Dense: $\mathrm{Var}[\hat{A}] \geq \sigma_R^2/G > 0$ — signal in all regimes

Proposition 2: Information-theoretic bound

$I(Q^*; I \mid R_{\text{coarse}}, a) > 0$ — quantifies the value of any asymmetric info

Proposition 3: Calibration safety

If $\mathcal{L}_{\text{cal}} = 0$: optimal policies under $R_{\text{self}} \subseteq$ optimal policies under R_{coarse}

Dense reward doesn't introduce false optima.

These are prove-once, apply-everywhere results. Most tractable theoretical contribution

Part III: Key Design Decisions

Why Reward Shaping, Not Distillation

Alternative: teacher generates corrected solution conditioned on I, student matches it via KL.

Three reasons reward shaping wins:

1. Asymmetric information creates **evaluative** advantage, not **generative** advantage. Seeing "IndexError at line 42" helps you *judge*, not *write*. The teacher is a better judge, not a better coder.

2. Distillation is mode-seeking, **collapses diversity**. Reward shaping defines a landscape the student explores freely. Multiple diverse solutions can all get high reward.

3. With shared weights, distillation target **moves every gradient step** (circular). Reward under stop-gradient is a fixed signal per rollout.

Why Not Just Use Opus/GPT-4 as Judge?

The argument was never about single-rollout judgment quality. Three properties an external judge can't have:

1. Cost at RLVR scale. $G \times B \times N = 100\text{K}+$ judge calls. EGSR is one forward pass in GPU memory already loaded. Zero marginal cost.

2. Distributional gap over training. Opus judges well at step 0. By step 200, the policy writes code Opus wasn't calibrated for. Evasion is documented (Gao et al. 2023). A parameter-aligned self-judge can't be evaded.

3. Practical self-containment. No API dependency, no inference budget, no external model. The training loop is fully self-contained.

Empirically: need EGSR to match or approach external judge on downstream Pass@1 at

"Isn't This Just Prompting?"

Honest answer: the teacher forward pass *is* prompting. If that were the whole contribution, it's a blog post.

The contribution is everything around the prompt:

1. Training integration. Replacing GRPO's reward signal, computing gradients through dense self-reward, jointly optimizing generation and judgment. Prompting doesn't have a backward pass.

2. Label-free maintenance. $\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$ shape shared representations to keep judgment calibrated across hundreds of RL steps while the model changes. Learning dynamics question, not prompting question.

3. Theoretical results. Variance reduction, MI bound, calibration safety are properties of the training algorithm, not the prompt.

On-Policy Precision and Self-Judge Hacking

On-policy claim: We say **parameter-aligned**, not "on-policy." Teacher sees privileged info (f_i) student never sees. This is a feature — judge with answer key. Key property: capabilities are identical and evolve in lockstep.

Self-judge hacking — three defenses:

1. $\mathcal{L}_{\text{cal}}$ prevents **global** miscalibration — systematic leniency directly increases loss
2. $\mathcal{L}_{\text{pred}}$ constrains **internal execution model** — a model that predicts IndexError can't be lenient about IndexError when judging (shared representations)
3. **Consistency monitoring** — track R_{self} vs error-severity heuristics over training

Residual risk: local leniency within calibrated aggregates. Measured in E4. Acknowledged honestly.

Test-Case Fraction Baseline

$R = k/n$ (fraction of tests passed) is the obvious dense reward that needs no self-judgment.

Don't avoid it — **add as explicit baseline** and show EGSR wins:

Fraction says *how much* failed. EGSR says *why*.

Key experiment: Bucket rollouts by identical pass fraction. Within the same bucket (e.g., both pass 8/10), show EGSR assigns different R_{self} that better predicts edit distance to correct solution.

Two rollouts with 8/10: one has a subtle off-by-one, one has a fundamentally wrong algorithm. Fraction can't distinguish them. EGSR reads the stack trace and can.

Also: many RLVR setups are single-verdict (AIME, one integration test). Fraction is

Part IV: Experimental Plan

Experiment Roadmap

Exp	Name	Weeks	Type
E0	Infrastructure validation	1	Setup gate
E1	Signal validation (go/no-go)	2–3	**HARD GATE**
E2	Dense reward effectiveness	4–8	Core result
E3	Judge maintenance	4–8	Core (most novel)
E4	Reward hacking dynamics	9–11	Supporting
E5	Scale + domain transfer	9–11	Supporting
E6	Mechanistic analysis	11–13	Bonus

E1 is the hard gate. If grounded AUC $\approx$ ungrounded AUC at 4B, thesis is dead.

E3 is the most novel. If label-free maintenance $\geq$ labeled buffer, that's standalone.

Total: 13 weeks. Framework: verl + FSDP2 + vLLM. Models: Qwen3-4B/14B, OLMo3-7B.

Key Ablations

Ablation	Experiment	Values
Judgment granularity	E1, E2	Binary vs 5-class
λ_1 (feedback pred.)	E3	$\{0.01, 0.1, 0.5, 1.0\}$
λ_2 (calibration)	E3	$\{0.01, 0.1, 0.5, 1.0\}$
Warm-start SFT	E1, E3	With vs without
Group size G	E2	$\{4, 8, 16\}$
w_{exploit}	E4	$\{-1.0, -0.5, -0.25, 0\}$

Highest-impact ablation: Does EGSR G=4 match GRPO G=16? If yes, 4× compute savings.

Part V: Problems Identified and Solutions

Comprehension vs. Generation Gap

Concern: $\mathcal{L}_{\text{pred}}$ maintains *comprehension* (understanding code). But solving problems requires *generation* (producing code). These activate overlapping but not identical circuits.

A model could maintain perfect feedback prediction while losing generation ability — like a chess player who evaluates positions perfectly but forgot their openings.

Three responses:

1. $\mathcal{L}_{\text{cal}}$ bridges the gap — judging one's own rollouts requires generation-adjacent representations
2. The Pass@k diagnostic resolves this empirically — if Pass@256 recovers, generation circuits are intact
3. If the gap is real, combine $\mathcal{L}_{\text{pred}}$ with self-generating replay (generate correct solutions, verify, SFT)

Framing Options (Decided)

Framing	Pitch	Assessment
1. Dense reward for RLVR	Solve sparse reward	Clean but narrow
2. **Asymmetric info framework**	**General self-reward paradigm**	**Highest ceiling (chosen)**
3. Learning from environment	Interact and learn	Vague, undersells mechanism
4. Reward hack detection	Self-judge as detector	Secondary result
5. Standard distillation	Teacher generates, student matches	Worse (see analysis)

Decision: Framework section is general (asymmetric information). Experiments validate RLVR instantiation. Future instantiations (RLHF, agents) noted in discussion. Title: "Self-Distilling Rewards from Environment Feedback." Don't oversell — validate one, note generality.

Part VI: Deliverables Completed

Documents Produced

Papers:

- Full NeurIPS paper scaffold (v2, 15 pages, all placeholders marked)
- Mini 5-page class project version (v2)
- Research proposal (10 pages)

Presentations:

- Proposal slides (v2, reframed, 20 slides)
- Paper slides (v2, with result placeholders, 26 slides)
- Asymmetric information framing slides (20 slides)

Infrastructure:

- Claude Project instructions at CLAUDE.md for Claude Code

NeurIPS Readiness Assessment

Is it novel? Partially. Individual ingredients exist. Combination is new. Label-free maintenance ($\mathcal{L}_{\text{pred}} + \mathcal{L}_{\text{cal}}$) is most genuinely novel.

Is it NeurIPS quality? Borderline — entirely contingent on empirical results. Strong results (8–10 pt Pass@1 gain, matching external judge, E3 showing label-free $\geq$ labeled): clear accept. Marginal results (2–3 pt gain): rejection as incremental.

Estimated acceptance probability: 40% base, rising to 70%+ if empirical story is strong on three axes: (1) within-bucket wins over fraction baseline, (2) E3 judge maintenance, (3) G=4 matching G=16 for compute efficiency.

What would make it clearly sufficient:

- E3 (judge maintenance) as standalone result
- Dense reward enabling smaller G (compute savings)

Part VII: Research Directions Explored

Backburner Ideas (Ranked)

Rank	Idea	Novel?	Status
1	Multi-objective RLVR (Lexicographic/Pareto GRPO)	Yes	Lexicographic GRPO is publishable unit. No prior work for LLMs.
2	Continual RLVR rehearsal ($\mathcal{L}_{\text{pred}}$ as maintenance)	Yes	Falls out of EGSR. Run diagnostic as EGSR extension.
3	Mixture of Distillations (Specialist RLVR + routing)	Partial	Solid but risks "just engineering." Per-token routing is novel piece.
4	Population RLVR (K seeds + cross-pollination)	Partial	Least crowded exploration idea. GLM-5 validates merging mechanism.
5	Diverse reward axes (Multi-axis self-judgment)	Yes	Natural EGSR extension. Phase 0 checks axis independence.
6	Latent perturbation	No	DLR is too close.
7	Discovery vs reweighting	No	Yue et al. + POPE + AEPO crowded.

Connections Between All Ideas

Root cause: RLVR narrows the policy distribution.

This causes three problems simultaneously:

- **Sparse reward** (no signal among failures) $\rightarrow$ EGSR
- **Mode collapse** (diversity lost) $\rightarrow$ Multi-objective / Population / Diverse rewards
- **Forgetting** (prior capabilities lost) $\rightarrow$ Continual rehearsal / Sleep / MoD

Any method that maintains distributional breadth addresses all three.

EGSR provides the foundation: dense reward is necessary for all downstream ideas. Multi-objective RLVR builds on dense reward (needs vector R, not scalar). Continual rehearsal uses $\mathcal{L}_{\text{pred}}$ from EGSR. Diverse rewards extend EGSR to K axes. Population RLVR benefits from EGSR's compute efficiency (G=4).

Priority Roadmap

Now (weeks 1–13):

1. Run E0 (infra validation)
2. Run E1 (signal validation — go/no-go)
3. If E1 passes: E2–E6, write paper, submit NeurIPS 2026

During EGSR experiments, cheap additions:

4. Continual learning diagnostic: after code RLVR, measure math Pass@k (free)
5. Add $\mathcal{L}_{\text{pred}}$ on math during code RLVR, measure retention (one extra condition)

After EGSR ships:

6. Population RLVR Phase 0: 4 seeds, measure coverage overlap (1 week)
7. Diagnostic Phase 0: K [illegible]